



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 07

No de Prácticas(s): Proyecto01

Integrante(s): 323078401

323115296

323082392

323239441

323253916

No. de lista o

brigada: 04

Semestre: 2027-I

Fecha de entrega: 28/09/2026

Observaciones: _____

CALIFICACIÓN: _____

Índice

1. Introducción	2
1.1. Planteamiento del problema	2
1.2. Motivación	2
1.3. Objetivos	2
2. Marco Teórico	2
3. Desarrollo	3
4. Resultados	4
4.1. Clase Enfermero	6
4.2. Clase Paciente	6
4.3. Clase Sistema	6
5. Conclusiones	6

1. Introducción

1.1. Planteamiento del problema

Se busca gestionar y simular el flujo de atención dentro de un entorno hospitalario. Consiste en administrar un registro preciso del personal médico y de enfermería junto con los pacientes que ingresan al sistema, garantizando que el flujo de atención respete reglas como la correspondencia de especialidades médicas, la limitación de la capacidad de pacientes por profesional y la correcta actualización de los estados clínicos individuales a lo largo de la simulación.

1.2. Motivación

La automatización de este proceso de simulación de un sistema hospitalario, esto permite que las reglas internas del hospital se evalúen algorítmicamente antes de cualquier asignación, optimizando la gestión interna y proporcionando un acceso rápido y ordenado al estado actualizado de todas las atenciones.

1.3. Objetivos

- Simular el registro y almacenamiento en memoria de médicos, enfermeros y pacientes.
- Restringir la capacidad simultánea de atención del personal de salud a límites fijos preestablecidos.
- Controlar la asignación de pacientes validando automáticamente la coincidencia de especialidades.
- Gestionar e identificar mediante reportes el estado médico puntual de un paciente.
- Procesar e imprimir listas de pacientes mediante un ordenamiento alfabético descendente.

2. Marco Teórico

Para la inicialización de objetos, el lenguaje Java emplea constructores, que son bloques de código invocados automáticamente al utilizar la palabra clave **new** para crear una instancia [1]. Dentro de la estructura de estos constructores, resulta indispensable la palabra reservada **this**, la cual proporciona una referencia directa al objeto actual en ejecución, permitiendo distinguir de manera explícita los atributos propios de la clase frente a los parámetros o variables locales que reciben el mismo nombre [2].

Por otro lado, la manipulación textual utiliza métodos nativos de la clase **String**. Entre ellos se encuentra el método `.equalsIgnoreCase()`, el cual compara el contenido de dos cadenas omitiendo la diferencia entre letras mayúsculas y minúsculas; y el método `.charAt(int)`, que extrae y devuelve el valor exacto del carácter ubicado en el índice numérico especificado [3].

Finalmente, para el manejo y estructuración de la información se emplean arreglos unidimensionales (estructuras lineales de tamaño fijo). Cuando es necesario presentar la información de dichos arreglos bajo un formato específico, se aplican algoritmos como

el Ordenamiento Burbuja (Bubble Sort). Este método clásico opera realizando pasadas sucesivas sobre la colección, comparando de manera adyacente los elementos y provocando un intercambio en sus posiciones en caso de no cumplir con la secuencia esperada, iterando hasta que la lista quede totalmente ordenada [4].

3. Desarrollo

El programa se integró en Java, compuesto por cinco clases, una de ellas siendo la clase **Main**, diseñado para simular la interacción y atención dentro de un hospital. Este utiliza arreglos unidimensionales de objetos para almacenar múltiples instancias de profesionales y pacientes, gestionando el ciclo de vida de los datos desde su creación en tiempo de ejecución. Cada entidad en el sistema cuenta con responsabilidades delimitadas y colabora con el resto solicitando validaciones, cruzando datos o alterando los estados de sus contrapartes.

La clase **Medico** tiene la responsabilidad principal de gestionar individualmente las consultas y recetar tratamientos. Al instanciar un objeto de esta clase mediante su constructor, recibe los parámetros de nombre, cédula y especialidad, los cuales son asignados directamente a sus atributos correspondientes utilizando la palabra reservada **this**. En su diseño interno, cada médico maneja un contador de pacientes y un arreglo llamado **pacientesAsignados** con una capacidad estricta de 10 posiciones. Para cumplir con la restricción de atención, implementa un atributo apuntador temporal llamado **pacienteEnConsulta**, garantizando que únicamente un paciente pueda estar vinculado activamente en una revisión simultánea. Cuando el médico requiere visualizar su estado, invoca un método que ejecuta un algoritmo de Ordenamiento Burbuja; para organizarlos alfabéticamente de forma descendente, el procesamiento compara las iniciales de los pacientes empleando el método `.charAt(0)` de la API de Java, evaluando sus valores numéricos y generando un intercambio en el arreglo si el carácter izquierdo es menor al derecho.

De manera complementaria, la clase **Enfermero** modela al personal de apoyo de la institución. Su proceso de instanciación, asignación de atributos con **this** y algoritmo de ordenamiento son prácticamente idénticos a los de la clase médica. Sin embargo, su estructura refleja responsabilidades diferentes: un enfermero no requiere del apuntador de paciente en consulta ya que su función exclusiva es administrar tratamientos. Asimismo, para reflejar las limitantes impuestas, el arreglo de **pacientesAsignados** de esta clase restringe su tamaño máximo a únicamente 3 personas por especialidad.

Por su parte, la clase **Paciente** actúa principalmente como una estructura de datos portadora de un estado médico. Su inicialización en el constructor requiere solamente el nombre y la especialidad solicitada. El procesamiento en esta clase se basa en el manejo de banderas lógicas mediante el uso de atributos booleanos explícitos, tales como **estaEnConsulta** y **estaEnTratamiento**. Estos atributos inician en falso y su valor únicamente se modifica cuando la instancia recibe una indicación externa (ya sea de un médico o un enfermero) pidiéndole transitar de su estado de espera a la fase clínica que corresponda.

La clase **Sistema** sirve como el controlador logístico central. Durante su instanciación genera arreglos globales y vacíos capaces de albergar hasta 50 médicos, 100 enfermeros y pacientes indefinidos, comenzando con 10 pacientes. Su responsabilidad recae en asegurar que la información cuadre lógicamente antes de procesar una atención. Al recibir una petición para asignar un paciente, esta clase evalúa los objetos utilizando el método

`.equalsIgnoreCase()` para garantizar que la especialidad demandada por el paciente coincida sin importar variaciones con la del profesional. Solo tras superar exitosamente esta verificación, el objeto del sistema contacta internamente al objeto del médico o enfermero, transmitiendo la petición de admisión para que estos validen su espacio en los arreglos locales y concreten la vinculación. Así mismo también amplía el tamaño de la lista de pacientes cuando el tamaño máximo se ha alcanzado: Crea una lista con el doble de capacidad que la actual y agrega a los pacientes que ya estaban registrados; y reemplaza a la lista vieja por la nueva que creó.

Finalmente, la clase `Main` se encarga de gestionar toda la interfaz con el usuario. Para su funcionamiento, crea una instancia de la clase `Scanner` y `Sistema`, habilitando la captura ininterrumpida de textos y comandos numéricos por consola, además del sistema del hospital al que se le asigna un nombre. La lógica de ejecución fluye de manera sostenida utilizando un ciclo `do-while` que envuelve una estructura de selección `switch`, mostrando menús diferenciados por perfil. Desde aquí, el programa recolecta los datos puros, instancia los nuevos objetos y solicita a la clase `Sistema` que los añada a sus registros, completando así la simulación integral del entorno hospitalario.

4. Resultados

```
=====
      SISTEMA DE GESTIÓN HOSPITALARIA
      Hospital: Hospital Central Universitario
=====
1. Médico
2. Enfermero
3. Paciente
4. Salir
Seleccione una opción: 1

--- MENÚ DE MÉDICO ---
1. Registrar médico
2. Solicitar paciente al sistema
3. Dar consulta
4. Dar tratamiento (recetar)
5. Ver lista de pacientes
6. Regresar
Opción: 1
Nombre del médico: Zayas Gonzalez Diego Raul
Cédula: 323078401
Especialidad: oncologia
Médico ZAYAS GONZALEZ DIEGO RAUL registrado exitosamente en el sistema.
```

Figura 1: En el menú principal y registro inicial de entidades médicas, de enfermería y pacientes en el simulador sigue el mismo concepto.

```

--- MENÚ DE PACIENTE ---
1. Registrar paciente
2. Asignar paciente a Médico
3. Asignar paciente a Enfermero
4. Solicitar consulta médica (automática)
5. Ver estado y tratamiento
6. Regresar
Opción: 2

Seleccione Paciente:
1. PERES PEREZ HECTOR (ONCOLOGIA)
1

Seleccione Médico:
1. ZAYAS GONZALEZ DIEGO RAUL (ONCOLOGIA) [0/10]
1
Paciente PERES PEREZ HECTOR admitido con el Dr./Dra. ZAYAS GONZALEZ DIEGO RAUL (1/10).

```

Figura 2: Demostrando la validación exitosa de asignación de pacientes evaluando la coincidencia de especialidad y advirtiéndolo cuando hay límite de cupo.

```

Seleccione Paciente:
1. ZAYAS GONZALEZ DIEGO RAUL
1
El Dr./Dra. PEREZ PEREZ JAVIER está dando consulta a: ZAYAS GONZALEZ DIEGO RAUL

--- MENÚ DE MÉDICO ---
1. Registrar médico
2. Solicitar paciente al sistema
3. Dar consulta
4. Dar tratamiento (recetar)
5. Ver lista de pacientes
6. Regresar
Opción: 4

Seleccione Médico:
1. PEREZ PEREZ JAVIER [1/10]
1
Seleccione Paciente:
1. ZAYAS GONZALEZ DIEGO RAUL
1
Ingrese indicación / receta médica: Radioterapia por 6 meses e intravenosa vernos depues para ver tumor
El Dr./Dra. PEREZ PEREZ JAVIER recetó a ZAYAS GONZALEZ DIEGO RAUL: Radioterapia por 6 meses e intravenosa
a vernos depues para ver tumor

```

Figura 3: Muestra la interacción médica y de enfermería, mostrando cómo un paciente transita de estado de espera a consulta, y posteriormente al recibir su tratamiento.

```

--- Lista de Pacientes: Dr./Dra. PEREZ PEREZ JAVIER ---
Especialidad: ONCOLOGIA | Total: 3/10
1. ZAYAS GONZALEZ DIEGO RAUL | Especialidad: ONCOLOGIA | Estado: En tratamiento (Radioterapia por 6 meses e intravenosa vernos depues para ver tumor)
2. ZAYAS GONZALEZ DIEGO RAUL | Especialidad: ONCOLOGIA | Estado: En tratamiento (Radioterapia por 6 meses e intravenosa vernos depues para ver tumor)
3. ZAYAS GONZALEZ DIEGO RAUL | Especialidad: ONCOLOGIA | Estado: En tratamiento (Radioterapia por 6 meses e intravenosa vernos depues para ver tumor)

```

Figura 4: Captura visualizando el correcto funcionamiento del algoritmo en la lista de pacientes de un profesional, exponiendo la información en un orden alfabético descendente.

En el siguiente enlace se encuentra disponible el archivo con el modelo gráfico Contar Palabras: [Ver imagen del Diagrama de clases.png en GitHub](#).

- En el diagrama se muestran clases en el que se van llamando las funciones de cada uno de los metodos y objetos llamandose e invocandose entre si para inicializar los datos del enfermero,medico y paciente en las demas diagramas).

Clase Medico

- **darConsulta:** Registra que el médico está atendiendo a un paciente en este momento.
[Enlace al diagrama de darConsulta](#)

4.1. Clase Enfermero

- **darTratamiento:** Administra la indicación o medicamento recetado al paciente.
[Enlace al diagrama de darTratamiento](#)

4.2. Clase Paciente

- **solicitarConsulta:** Pide al sistema que le asigne atención médica de forma automática.
[Enlace al diagrama de solicitarConsulta](#)

4.3. Clase Sistema

- **asignarPaciente:** Vincula a un paciente disponible con un médico o enfermero de su misma especialidad.
[Enlace al diagrama de asignarPaciente](#)

5. Conclusiones

Los resultados obtenidos en las pruebas confirman que la aplicación directa del paradigma de Clases y Objetos resultó esencial para construir una simulación de la gestión hospitalaria. Al utilizar constructores, permitió que cada profesional contara con sus propias validaciones independientes para no exceder los límites de atención correspondientes a su nivel de responsabilidad.

Asimismo, la combinación de las estructuras de arreglos estáticos con funcionalidades de la API de Java permitió realizar la verificación clínica de forma automatizada, asegurando que el flujo clínico del paciente no sufriera anomalías. Finalmente, el diseño de la solución comprobó que algoritmos tradicionales como el Ordenamiento Burbuja siguen siendo funcionales; su aplicación concreta fue relevante para organizar los registros y presentar las atenciones requeridas.

Referencias

- [1] Oracle, “Providing Constructors for Your Classes,” *The Java Tutorials*, 2021. [Online]. Available: <https://docs.oracle.com/javase/tutorial/java/java00/constructors.html>
- [2] Oracle, “Using the this Keyword,” *The Java Tutorials*, 2021. [Online]. Available: <https://docs.oracle.com/javase/tutorial/java/java00/thiskey.html>

- [3] Oracle, “String (Java SE 17 & JDK 17),” *Oracle Help Center*, 2021. [Online]. Available: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html>
- [4] GeeksforGeeks, “Java Program for Bubble Sort,” 2023. [Online]. Available: <https://www.geeksforgeeks.org/java-program-for-bubble-sort/>